

want for creating computations that perform I/O. If we like the metaphor of lists or streams, we can design a list-like API for expressing I/O computations. If we discover some other composable abstraction, we can find a way of using that instead. Functional programming gives us that flexibility.

13.8 Summary

This chapter introduced a simple model for handling external effects and I/O in a purely functional way. We began with a discussion of factoring effects and demonstrated how effects can be moved to the outer layers of a program. We generalized this to an `IO` data type that lets us describe interactions with the outside world without resorting to side effects.

We discovered that monads in Scala suffer from a stack overflow problem, and we solved it in a general way using the technique of *trampolining*. This led us to the even more general idea of *free monads*, which we employed to write a very capable `IO` monad with an interpreter that used non-blocking asynchronous I/O internally.

The `IO` monad is not the final word in writing effectful programs. It's important because it represents a kind of lowest common denominator when interacting with the external world. But in practice, we want to use `IO` directly as little as possible because `IO` programs tend to be monolithic and have limited reuse. In chapter 15, we'll discuss how to build nicer, more composable, more reusable abstractions using essentially the same technique that we used here.

Before getting to that, we'll apply what we've learned so far to fill in the other missing piece of the puzzle: *local effects*. At various places throughout this book, we've made use of local mutation rather casually, with the assumption that these effects weren't *observable*. In the next chapter, we'll explore what this means in more detail, see more examples of using local effects, and show how *effect scoping* can be enforced by the type system.